# Blockchain Exp 6

**Name:** Rohan Lalchandani **Class:** D20A
**Roll no:** 31 **Batch:** A

**Aim:** Creating Smart Contracts and performing transactions using Solidity and Remix IDE

**Theory:**

**Q1: What is a Smart Contract?**
A smart contract is a self-executing digital agreement stored on a blockchain. It automatically runs when certain conditions written in the code are met. Smart contracts do not require any middleman like a bank or lawyer because the rules and execution are controlled by the program itself. Once the contract is deployed on the blockchain, it cannot be easily changed and all transactions are recorded permanently. This makes the process transparent, secure, and reliable.

**Q2: Significance of smart Contracts in Ethereum Blockchain**
Smart contracts play a very important role in the Ethereum blockchain.

- **Automation:** Transactions and agreements are executed automatically when conditions are met.
- **No Intermediaries:** Removes the need for third parties like banks or agents. Transparency: All contract details and transactions are recorded on the blockchain and can be verified.
- **Security:** Data stored on the blockchain is difficult to alter, making contracts more secure.
- **Cost Efficient:** Reduces transaction and processing costs by eliminating middlemen.
- **Used in Many Applications:** Examples include voting systems, financial services (DeFi), supply chain management, and digital agreements.

**Implementation:**

**Rental Agreement System:**

1. **Smart Contract: RentalAgreement Contract**

The RentalAgreement contract is used to create and manage the rental agreement between the landlord and the tenant. When the contract is deployed on the blockchain using Remix IDE, the landlord initializes it by providing the tenant's Ethereum address, rent amount, and duration in months. Once deployed, the contract permanently stores these details in a secure and immutable manner. The contract ensures that both parties are clearly defined and that the agreement remains active until it is either completed or terminated by the landlord. Functions like getDetails() allow users to view agreement information at any time, ensuring transparency. The terminateAgreement() function gives control to the landlord to end the contract when required. This contract acts as the core agreement layer, ensuring that all rental terms are recorded and cannot be altered, thereby eliminating disputes and increasing trust between both parties.

Stores:

- Landlord address
- Tenant address
- Rent amount
- Duration (months)

Controls:

- Agreement status (active/inactive)

## Code:

```
contract RentalAgreement {

    address public landlord;
    address public tenant;
    uint public rentAmount;
    uint public durationInMonths;

    bool public isActive;

    constructor(address _tenant, uint _rentAmount, uint _durationInMonths) {
        landlord = msg.sender;
        tenant = _tenant;
        rentAmount = _rentAmount;
        durationInMonths = _durationInMonths;
        isActive = true;
    }

    function getDetails() public view returns (
        address, address, uint, uint, bool
    ) {
        return (landlord, tenant, rentAmount, durationInMonths, isActive);
    }

    function terminateAgreement() public {
        require(msg.sender == landlord, "Only landlord can terminate");
        isActive = false;
    }
}
```

### 2. Smart contract: PaymentManager Contract

The PaymentManager contract is used to handle all financial transactions related to the rental agreement. After deployment, it stores the rent amount and the landlord's address. The tenant interacts with this contract to pay rent using the payRent() function by sending the exact rent amount in cryptocurrency (wei). The contract automatically verifies the payment amount and transfers it directly to the landlord, removing the need for intermediaries. It also keeps a record of all payments made by each tenant

using a mapping, which can be accessed through the checkPayment() function. This ensures transparency and accountability, as both landlord and tenant can verify payment history at any time. Overall, this contract acts as the transaction layer, securely managing and automating rent payments on the blockchain.

Handles:

- Rent payment
- Transfer of money
- Payment tracking

**Code:**

```
contract PaymentManager {

    address public landlord;
    uint public rentAmount;

    mapping(address => uint) public totalPaid;

    constructor(uint _rentAmount) {
        landlord = msg.sender;
        rentAmount = _rentAmount;
    }

    function payRent() public payable {
        require(msg.value == rentAmount, "Incorrect rent amount");

        totalPaid[msg.sender] += msg.value;

        payable(landlord).transfer(msg.value);
    }

    function checkPayment(address tenant) public view returns (uint) {
        return totalPaid[tenant];
    }
}
```

# PROCEDURE:

Step 1: Open Remix IDE
Open the Remix Integrated Development Environment in the browser. Remix IDE is used to write, compile, deploy, and test Solidity smart contracts. https://remix.ethereum.org

Step2: Create a new file rental.sol, this will contain both the contracts
Compile and deploy both the contracts

Step 3: Testing the code

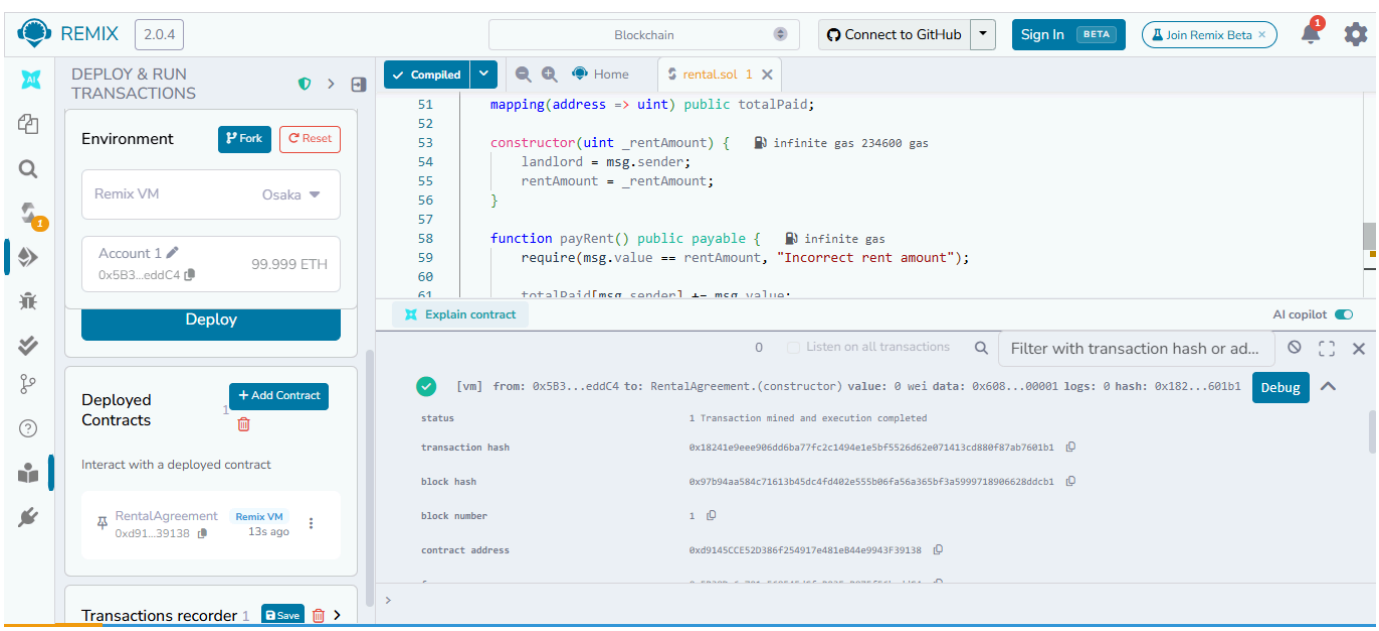
## Test 1: Deployment of RentalAgreement

### Input Values:

- Tenant Address → Account 2
- Rent Amount → 100
- Duration → 12

### Expected Output:

- Contract deployed successfully
- Landlord = Account 1
- Tenant = Account 2
- Status = Active (true)



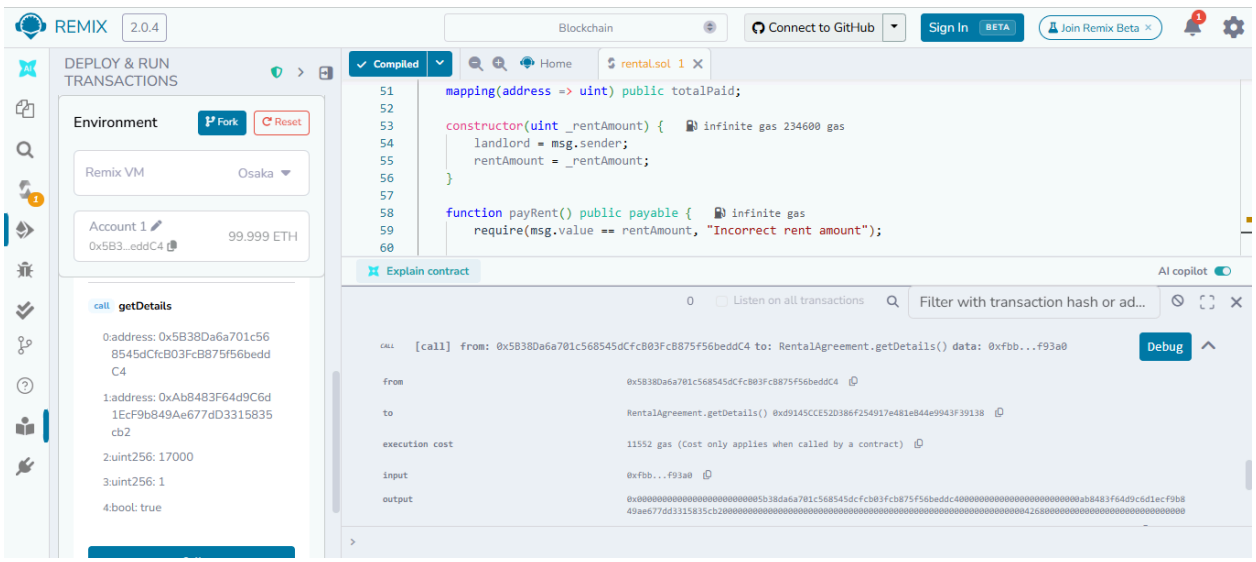
# Test2: View Agreement Details

## Function:

getDetails()

## Expected Output:

- Correct landlord address
- Correct tenant address
- Rent amount = 100
- Duration = 12
- Status = true




# Test 3: Deployment of PaymentManager

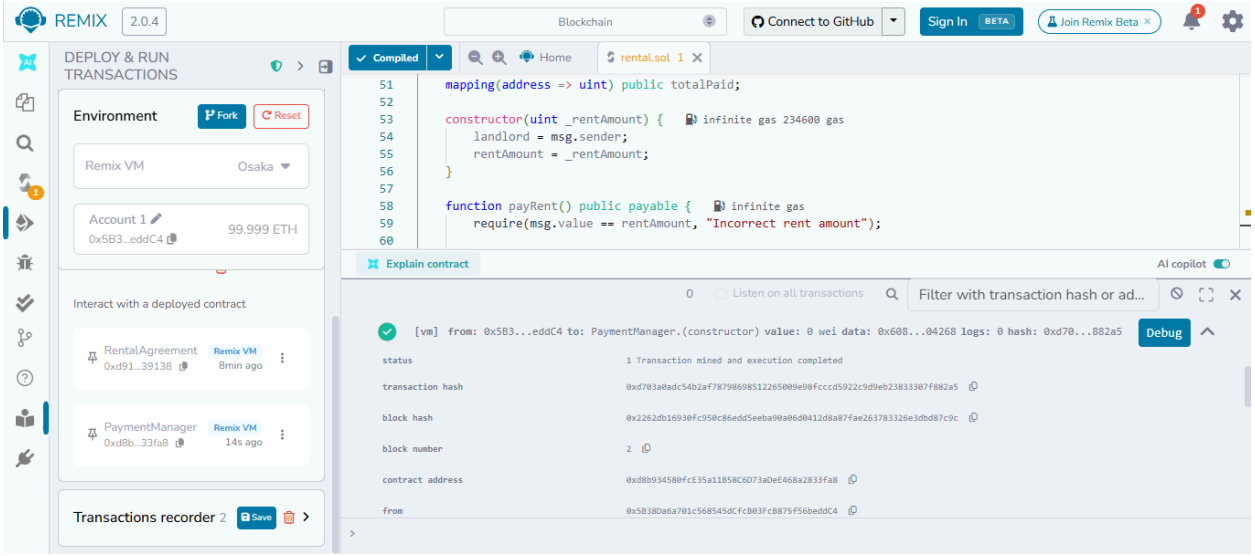
## 🔹 Input Value:

- Rent Amount → 17000

Deploy contract using Account 1

## Expected Output:

- Contract deployed successfully
- Landlord = Account 1




# Test 4: Successful Rent Payment

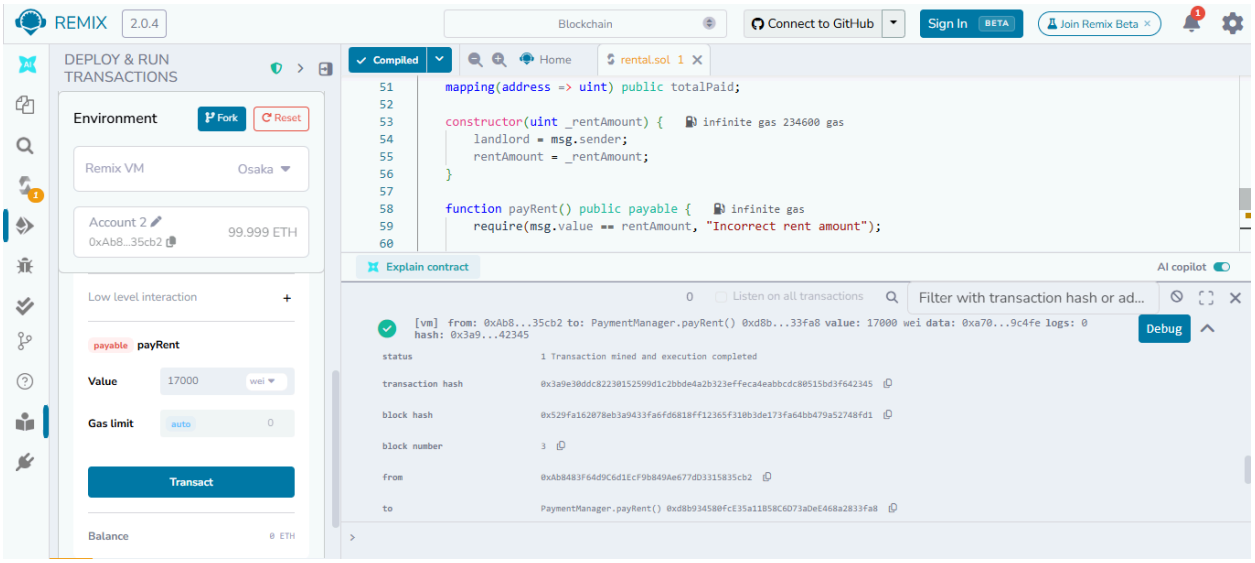
## Input:

- Account → Tenant (Account 2)
- Value → 100 wei

1. Switch to Account 2
2. Enter Value = 100
3. Call payRent()

## Expected Output:

- Transaction successful
- Amount transferred to landlord
- Payment recorded



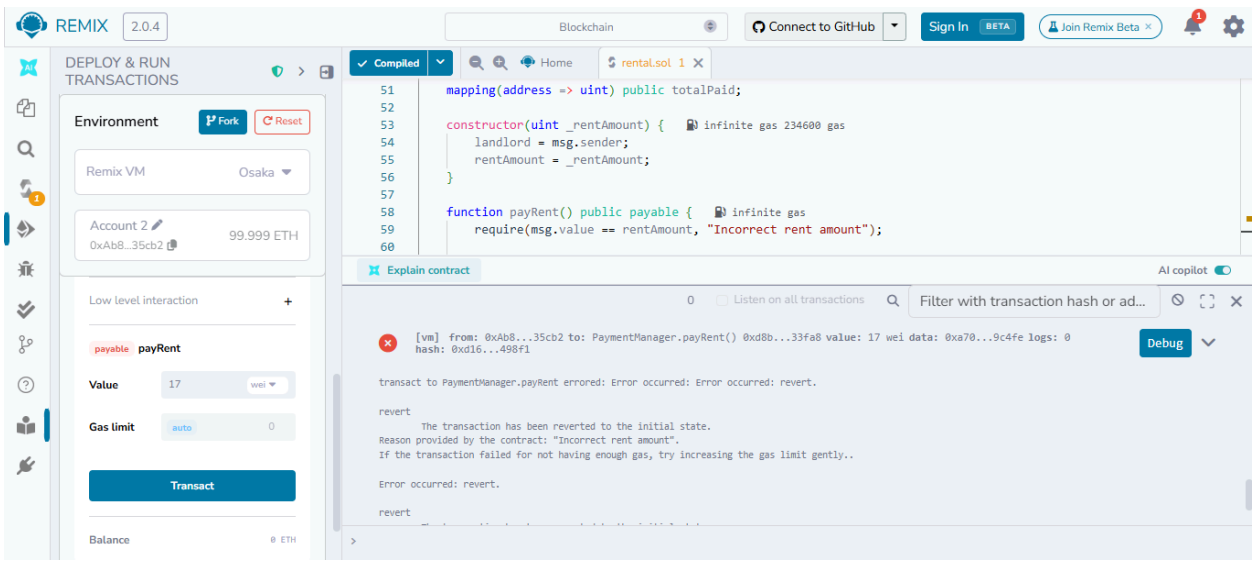
# Test 5: Incorrect Rent Payment

## Input:

- Value → 50 wei
1. Call payRent()

## Expected Output:

- Transaction should fail
- Error: "Incorrect rent amount"



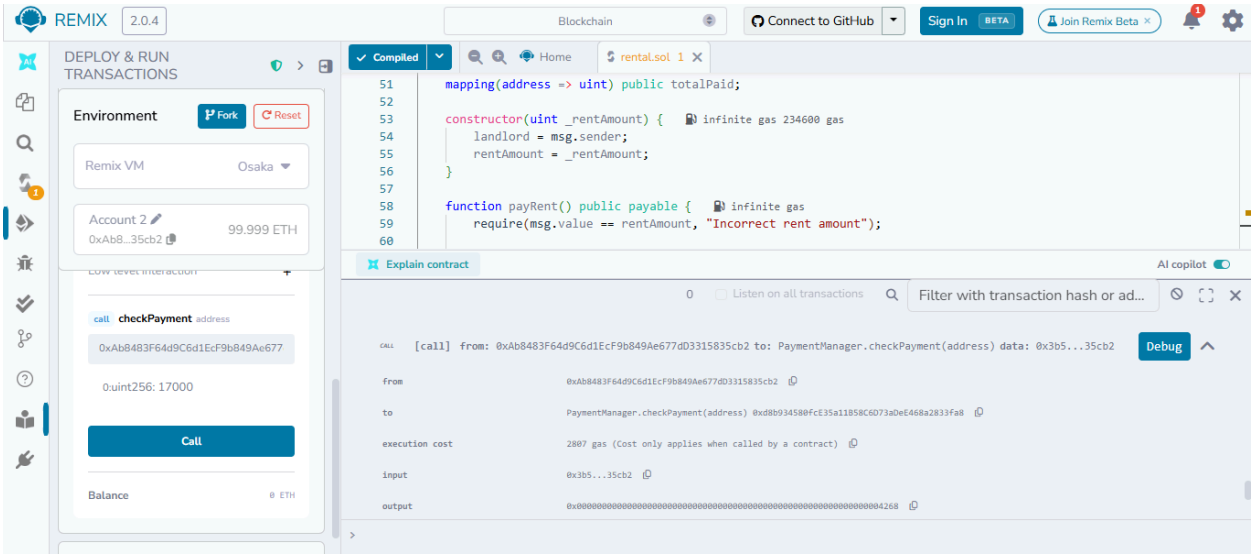

# Test 6: Check Payment Record

## 🔹 Function:

checkPayment(tenant address)

## 🔹 Expected Output:

- Returns 17000




# Test 7: Unauthorized Termination Attempt

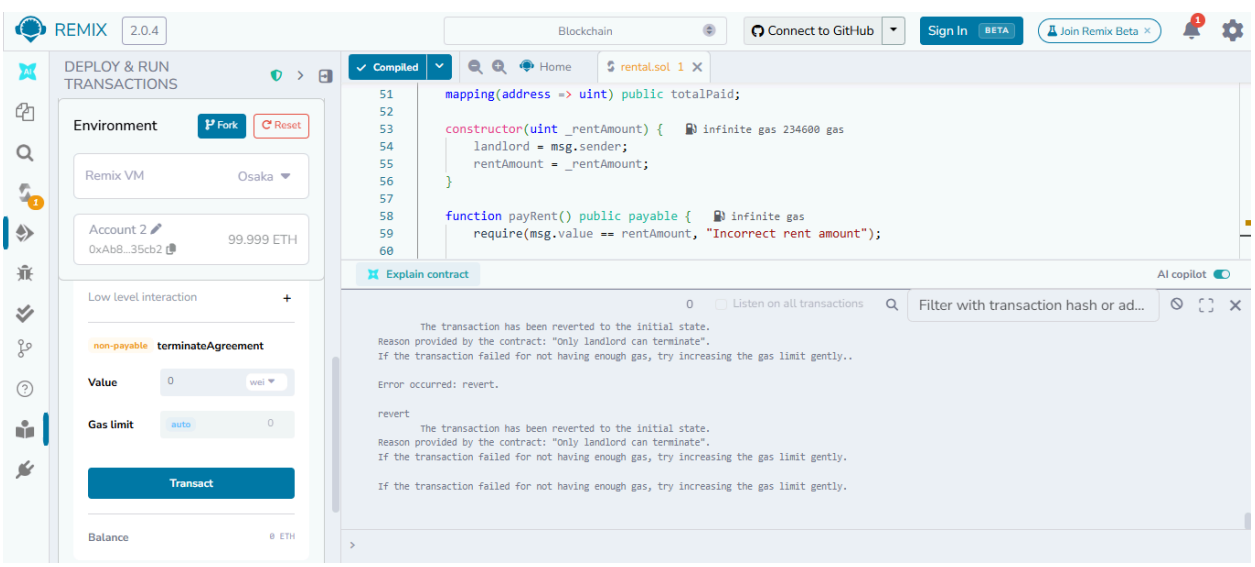
## Input:

- Caller → Tenant

1. Try calling terminateAgreement()

## 🔹 Expected Output:

- Transaction fails
- Error: "Only landlord can terminate"



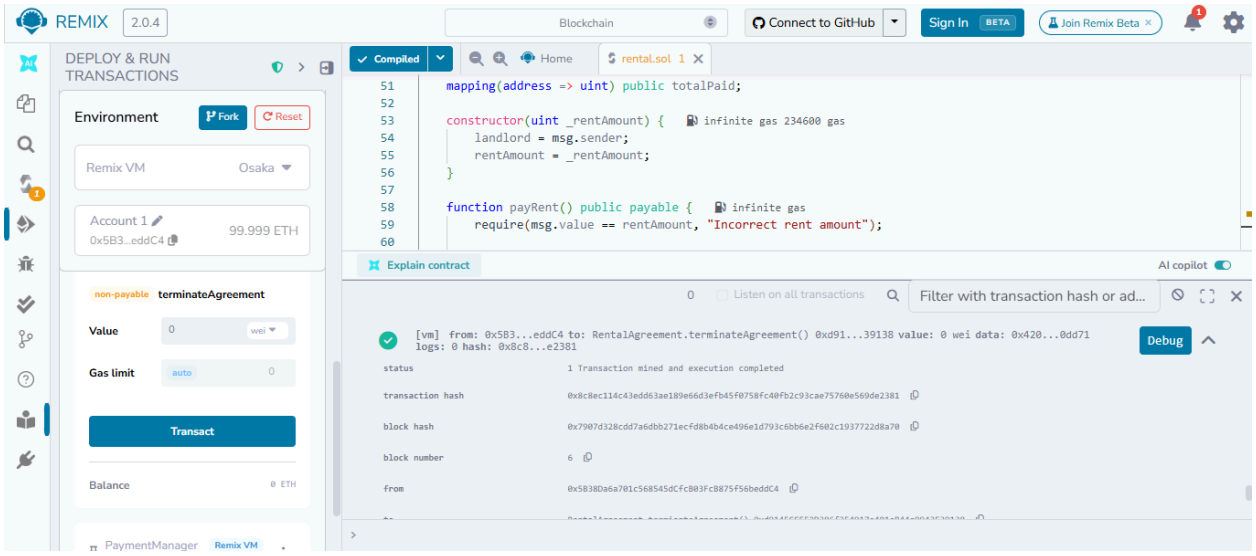
## Test 7: Agreement Termination by Landlord

### Input:

- Caller → Landlord
1. Call terminateAgreement()

### 🔹 Expected Output:

- isActive = false




## Conclusion:

Two smart contracts were developed: one for managing rental agreement details and another for handling rent payments. These contracts ensured that all operations, such as agreement creation, rent transactions, and termination, were executed automatically without the need for intermediaries. The use of blockchain ensured that all records are immutable, secure, and easily verifiable.

Through testing, it was observed that the system correctly handled valid and invalid transactions, enforced access control, and maintained accurate records of payments. The implementation also highlighted the advantages of decentralized systems in reducing fraud, increasing trust, and automating processes.